

INFORME DE AUDITORÍA TÉCNICA DE SEGURIDAD

Evaluación de vulnerabilidades OWASP Top 10 | Plataforma WebGoat 8.1.0



Preparado para: **Departamento de Ciberseguridad - KeepCoding**

Asignatura: **Introducción a la Ciberseguridad 101**

Analista: Nicolás Navares

Fecha: 25 de enero de 2026

CONFIDENCIALIDAD Y AVISO LEGAL

Aviso de Confidencialidad Este documento es de carácter estrictamente confidencial. Contiene información detallada sobre vulnerabilidades de seguridad descubiertas durante la auditoría de la plataforma WebGoat 8.1.0. El acceso a este informe está restringido exclusivamente a los evaluadores de la práctica. Queda prohibida la reproducción total o parcial de este informe sin autorización de la organización.

Exención de Responsabilidad Este informe detalla pruebas realizadas en un entorno de laboratorio controlado y aislado (Docker). El autor no se responsabiliza del uso de estas técnicas o conocimientos en sistemas reales sin la debida autorización. La seguridad es un estado dinámico y este documento refleja exclusivamente los hallazgos encontrados en la fecha de emisión.

ÍNDICE

1. RESUMEN EJECUTIVO	04
1.1 Resumen del proceso realizado	
1.2 Hallazgos y Análisis de Riesgo	
1.3 Recomendación Principal	
2. VISIÓN GENERAL DE LA AUDITORÍA	05
2.1 Metodología aplicada	
2.2 Herramientas de análisis	
2.3 Ámbito	
3. FASE DE RECONOCIMIENTO Y ANÁLISIS	06
3.1 Escaneo de servicios e infraestructura	
4. ANÁLISIS TÉCNICO DE VULNERABILIDADES	07
4.1 A3: Inyección SQL - Impacto en la Confidencialidad	07-08
4.2 A3: Inyección SQL - Impacto en la Integridad (Query Chaining)	09-10
4.3 A3: Cross-Site Scripting (XSS Reflejado)	11
4.4 A5: Security Misconfiguration (XXE Injection)	12-13
4.5 A6: Componentes Vulnerables y Obsoletos	14
4.6 A7: Fallos de Identificación y Autenticación	15
5. CONCLUSIONES Y CIERRE	16
5.1 Cuadro resumen de riesgos	
5.2 Recomendaciones de remediación	
5.3 Análisis de Post-explotación	
6. REFERENCIAS Y FUENTES TÉCNICAS	17

1. RESUMEN EJECUTIVO

1.1 Resumen del proceso realizado Se ha llevado a cabo una auditoría técnica de seguridad sobre la plataforma **WebGoat 8.1.0** con el objetivo de evaluar su resistencia ante los vectores de ataque más comunes descritos en el estándar **OWASP Top 10**. El proceso se ha realizado bajo una metodología de "**Caja Negra**" (**Black Box**), analizando la aplicación desde la perspectiva de un atacante externo sin acceso previo al código fuente ni a credenciales privilegiadas.

1.2 Hallazgos y Análisis de Riesgo Durante las pruebas se detectaron fallos críticos que comprometen la tríada de la seguridad de la información (Confidencialidad, Integridad y Disponibilidad).

- **Confidencialidad:** Comprometida mediante Inyección SQL y XXE, permitiendo la extracción de bases de datos completas y archivos del servidor.
- **Integridad:** Comprometida mediante "Query Chaining", permitiendo la alteración no autorizada de registros financieros (salarios).
- **Control de Acceso:** Comprometido debido a políticas de contraseñas débiles que facilitan el robo de cuentas administrativas.

1.3 Recomendación Principal La aplicación requiere una remediación urgente centrada en la **validación estricta de entradas** ("Input Validation"). No se debe confiar en los datos proporcionados por el usuario; estos deben ser sanitizados y procesados mediante tecnologías seguras (como consultas parametrizadas) antes de interactuar con el servidor o la base de datos.

2. VISIÓN GENERAL DE LA AUDITORÍA

La presente auditoría técnica tiene como finalidad identificar brechas de seguridad explotables en la infraestructura y lógica de la aplicación WebGoat. A continuación se detallan los métodos y herramientas empleados.

2.1 Metodología aplicada Se ha utilizado una metodología estructurada en fases secuenciales para garantizar un análisis exhaustivo:

1. **Reconocimiento:** Identificación activa de servicios, puertos y tecnologías subyacentes.
2. **Análisis de Vulnerabilidades:** Detección de puntos de entrada débiles en formularios y parámetros URL.
3. **Explotación (PoC):** Validación manual de las vulnerabilidades mediante "Pruebas de Concepto" para confirmar el riesgo real sin dañar el servicio.
4. **Informe y Remediación:** Documentación técnica de los hallazgos y propuesta de soluciones.

2.2 Herramientas de análisis

- **Nmap:** Escaneo de red, detección de puertos y fingerprinting del sistema operativo.
- **Burp Suite (Community Edition):** Proxy de interceptación para analizar y modificar peticiones HTTP/XML en tiempo real.
- **Navegador Web (DevTools):** Interacción directa con la aplicación y análisis de scripts del lado del cliente.
- **Terminal Linux:** Ejecución de scripts y herramientas de sistema.

2.3 Ámbito El análisis se limita estrictamente a la instancia local de WebGoat 8.1.0 desplegada en un contenedor Docker en la dirección **IP 127.0.0.1**, puertos **8080** y **9090**.

3. FASE DE RECONOCIMIENTO Y ANÁLISIS

El primer paso de la auditoría consistió en identificar la superficie de ataque disponible. Se utilizó la herramienta **Nmap** para enumerar los puertos abiertos y los servicios en ejecución.

3.1 Escaneo de servicios e infraestructura Se ejecutó el siguiente comando para analizar el host objetivo: **nmap -p 8080,9090 -sV -sC -A 127.0.0.1**

Resultados obtenidos:

- **Puerto 8080/tcp:** Servicio HTTP activo (identificado como Apache Tomcat). Es el punto de entrada principal de la aplicación web.
- **Puerto 9090/tcp:** Servicio de administración secundario.
- **Sistema Operativo:** Entorno basado en Linux (Kernel 2.6.x - 5.x).

La identificación de versiones específicas de software (Apache Tomcat) es crítica, ya que permitiría a un atacante buscar vulnerabilidades públicas (CVEs) asociadas a esas versiones antes de intentar cualquier ataque manual.

```
(n0v4r3k@N0v4r3k)~  
$ nmap -O 127.0.0.1  
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-19 19:31 WET  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.000019s latency).  
Not shown: 998 closed tcp ports (reset)  
PORT      STATE SERVICE  
8080/tcp  open  http-proxy  
9090/tcp  open  zeus-admin  
Device type: general purpose  
Running: Linux 2.6.X|5.X  
OS CPE: cpe:/o:linux:linux_kernel:2.6.32 cpe:/o:linux:linux_kernel:5 cpe:/o:linux:linux_kernel:6  
OS details: Linux 2.6.32, Linux 5.0 - 6.2  
Network Distance: 0 hops  
  
OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 1.66 seconds  
  
(n0v4r3k@N0v4r3k)~
```

4. ANÁLISIS TÉCNICO DE VULNERABILIDADES

4.1 A3: Inyección SQL - Impacto en la Confidencialidad

Descripción del Fallo La aplicación construye consultas a la base de datos concatenando directamente el texto que introduce el usuario. Esto permite inyectar código SQL arbitrario.

Proceso de Explotación En el módulo de búsqueda de empleados, se identificó que el campo "Last Name" no sanitiza la entrada. Se introdujo el payload de tautología: ' OR 1=1 --

Este código cierra la **comilla** original de la consulta, añade una condición que siempre es verdadera (**1=1**) y comenta el resto de la línea (--), forzando a la base de datos a devolver todos los registros existentes.

1 2 3 4 5 6 7 8 9 10 11 12 13

Compromising confidentiality with String SQL injection

If a system is vulnerable to SQL injections, aspects of that system's CIA triad can be easily compromised (if you are unfamiliar with the CIA triad, check out the CIA triad lesson in the general category). In the following three lessons you will learn how to compromise each aspect of the CIA triad using techniques like SQL string injections or query chaining.

In this lesson we will look at **confidentiality**. Confidentiality can be easily compromised by an attacker using SQL injection; for example, successful SQL injection can allow the attacker to read sensitive data like credit card numbers from a database.

What is String SQL injection?

If an application builds SQL queries simply by concatenating user supplied strings to the query, the application is likely very susceptible to String SQL injection.

More specifically, if a user supplied string simply gets concatenated to a SQL query without any sanitization or preparation, then you may be able to modify the query's behavior by simply inserting quotation marks into an input field. For example, you could end the string parameter with quotation marks and input your own SQL after that.

It is your turn!

You are an employee named John **Smith** working for a big company. The company has an internal system that allows all employees to see their own internal data such as the department they work in and their salary.

The system requires the employees to use a unique *authentication TAN* to view their data.

Your current TAN is **3SL99A**.

Since you always have the urge to be the most highly paid employee, you want to exploit the system so that instead of viewing your own internal data, you want to take a look at the data of all your colleagues to check their current salaries.

Use the form below and try to retrieve all employee data from the **employees** table. You should not need to know any specific names or TANs to get the information you need.

You already found out that the query performing your request looks like this:

```
"SELECT * FROM employees WHERE last_name = '' + name + '' AND auth_tan = '' + auth_tan + ''";
```

Employee Name:

Authentication TAN:

Sorry the solution is not correct, please try again.

malformed string: '' AND auth_tan = ''

1 2 3 4 5 6 7 8 9 10 11 12 13

Compromising confidentiality with String SQL injection

If a system is vulnerable to SQL injections, aspects of that system's CIA triad can be easily compromised (if you are unfamiliar with the CIA triad, check out the CIA triad lesson in the general category). In the following three lessons you will learn how to compromise each aspect of the CIA triad using techniques like *SQL string injections* or *query chaining*.

In this lesson we will look at **confidentiality**. Confidentiality can be easily compromised by an attacker using SQL injection; for example, successful SQL injection can allow the attacker to read sensitive data like credit card numbers from a database.

What is String SQL injection?

If an application builds SQL queries simply by concatenating user supplied strings to the query, the application is likely very susceptible to String SQL injection. More specifically, if a user supplied string simply gets concatenated to a SQL query without any sanitization or preparation, then you may be able to modify the query's behavior by simply inserting quotation marks into an input field. For example, you could end the string parameter with quotation marks and input your own SQL after that.

It is your turn!

You are an employee named John Smith working for a big company. The company has an internal system that allows all employees to see their own internal data such as the department they work in and their salary. The system requires the employees to use a unique authentication TAN to view their data. Your current TAN is 3SL99A.

Since you always have the urge to be the most highly paid employee, you want to exploit the system so that instead of viewing your own internal data, you want to take a look at the data of all your colleagues to check their current salaries. Use the form below and try to retrieve all employee data from the **employees** table. You should not need to know any specific names or TANs to get the information you need. You already found out that the query performing your request looks like this:

```
"SELECT * FROM employees WHERE last_name = '' + name + '' AND auth_tan = '' + auth_tan + ''";
```

✓

Employee Name: 'OR 1=1 --

Authentication TAN: TAN

Get department

You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
32147	Paulina	Travers	Accounting	46000	P45JSI
34477	Abraham	Holman	Development	50000	UU2ALK
37648	John	Smith	Marketing	64350	3SL99A
89762	Tobi	Barnett	Development	77000	TA9LL1
96134	Bob	Franco	Marketing	83700	LO9S2V

Figura 4.1: Explotación exitosa. El payload ' OR 1=1 -- revela la lista completa de empleados y sus datos privados.

Criticidad: CRÍTICA Este fallo rompe totalmente la **confidencialidad**. Un atacante puede descargar la base de datos entera de la organización, incluyendo datos personales (PII), salarios y credenciales de acceso.

Mitigación y Recomendación

- **Técnica:** Utilizar **Consultas Preparadas (Prepared Statements)** en el código Java.
- **Explicación:** Se debe separar la estructura de la consulta de los datos. Al usar *Prepared Statements*, la base de datos trata la entrada del usuario estrictamente como texto, impidiendo que se ejecute como código lógico.
- **Referencia OWASP:** [SQL Injection Prevention Cheat Sheet](#)

4.2 A3: Inyección SQL - Impacto en la Integridad (Query Chaining)

Descripción del Fallo El sistema permite el "Query Chaining" (encadenamiento de consultas), lo que significa que se pueden ejecutar múltiples instrucciones SQL separadas por un punto y coma (;) en una sola petición.

Proceso de Explotación El objetivo fue modificar el salario del empleado "Smith" sin autorización.

1. Estado Inicial: Se verifica el formulario y el salario actual.

1 2 3 4 5 6 7 8 9 10 11 12 13

Compromising Integrity with Query chaining

After compromising the confidentiality of data in the previous lesson, this time we are gonna compromise the **integrity** of data by using SQL **query chaining**.

If a severe enough vulnerability exists, SQL injection may be used to compromise the integrity of any data in the database. Successful SQL injection may allow an attacker to change information that he should not even be able to access.

What is SQL query chaining?

Query chaining is exactly what it sounds like. With query chaining, you try to append one or more queries to the end of the actual query. You can do this by using the ; metacharacter. A ; marks the end of a SQL statement; it allows one to start another query right after the initial query without the need to even start a new line.

It is your turn!

You just found out that Tobì and Bob both seem to earn more money than you! Of course you cannot leave it at that. Better go and *change your own salary so you are earning the most!*

Remember: Your name is John **Smith** and your current TAN is **3SL99A**.

Employee Name:	
Authentication TAN:	
Get department	

Still not earning enough or only your own salary must be changed! Better try again and change that.

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
96134	Bob	Franco	Marketing	83700	LO9S2V
89762	Tobi	Barnett	Development	77000	TA9LL1
37648	John	Smith	Marketing	64350	3SL99A
34477	Abraham	Holman	Development	50000	UU2ALK
32147	Paulina	Travers	Accounting	46000	P45JSI

Ejecución del Ataque: Se inyectó el siguiente payload para actualizar la tabla de empleados:
Smith' ; UPDATE employees SET salary = 9000000 WHERE last_name = 'Smith' --

1 2 3 4 5 6 7 8 9 10 11 12 13

Compromising Integrity with Query chaining

After compromising the confidentiality of data in the previous lesson, this time we are gonna compromise the **integrity** of data by using **SQL query chaining**.

If a severe enough vulnerability exists, SQL injection may be used to compromise the integrity of any data in the database. Successful SQL injection may allow an attacker to change information that he should not even be able to access.

What is SQL query chaining?

Query chaining is exactly what it sounds like. With query chaining, you try to append one or more queries to the end of the actual query. You can do this by using the ; metacharacter. A ; marks the end of a SQL statement; it allows one to start another query right after the initial query without the need to even start a new line.

It is your turn!

You just found out that Tobí and Bob both seem to earn more money than you! Of course you cannot leave it at that. Better go and change your own salary so you are earning the most!

Remember: Your name is John **Smith** and your current TAN is **3SL99A**.

✓

Employee Name:

Authentication TAN:

Well done! Now you are earning the most money. And at the same time you successfully compromised the integrity of data by changing your salary!

USERID	FIRST_NAME	LAST_NAME	DEPARTMENT	SALARY	AUTH_TAN
37648	John	Smith	Marketing	9000000	3SL99A
96134	Bob	Franco	Marketing	83700	LO9S2V
89762	Tobi	Barnett	Development	77000	TA9LL1
34477	Abraham	Holman	Development	50000	UU2ALK
32147	Paulina	Travers	Accounting	46000	P45JSI

Figura 4.2.2: Ataque exitoso. Se observa la confirmación del sistema y el cambio de salario en la base de datos.

Criticidad: ALTA / CRÍTICA Afecta a la **integridad** de los datos. Permite fraudes financieros, alteración de nóminas, borrado de tablas o creación de usuarios administradores "puerta trasera".

Mitigación y Recomendación

- **Técnica:** Deshabilitar la opción de **Multi-queries** en la configuración del driver de la base de datos y aplicar el principio de mínimo privilegio.
- **Explicación:** El usuario de base de datos que usa la web solo debería tener permisos de lectura (SELECT), nunca de escritura o modificación (UPDATE/DELETE) en formularios de búsqueda.
- **Referencia OWASP:** [SQL Injection Prevention Cheat Sheet](#)

4.3 A3: Cross-Site Scripting (XSS Reflejado)

Descripción del Fallo La aplicación refleja los datos introducidos por el usuario en el navegador sin codificarlos correctamente. Esto permite la ejecución de scripts maliciosos (JavaScript).

Proceso de Explotación En el campo de "Credit Card Number" del carrito de compra, se inyectó el siguiente script de prueba: `<script>alert('TE PILLE')</script>`

Al procesar la compra, el navegador interpretó el script y ejecutó la ventana de alerta, confirmando la vulnerabilidad.

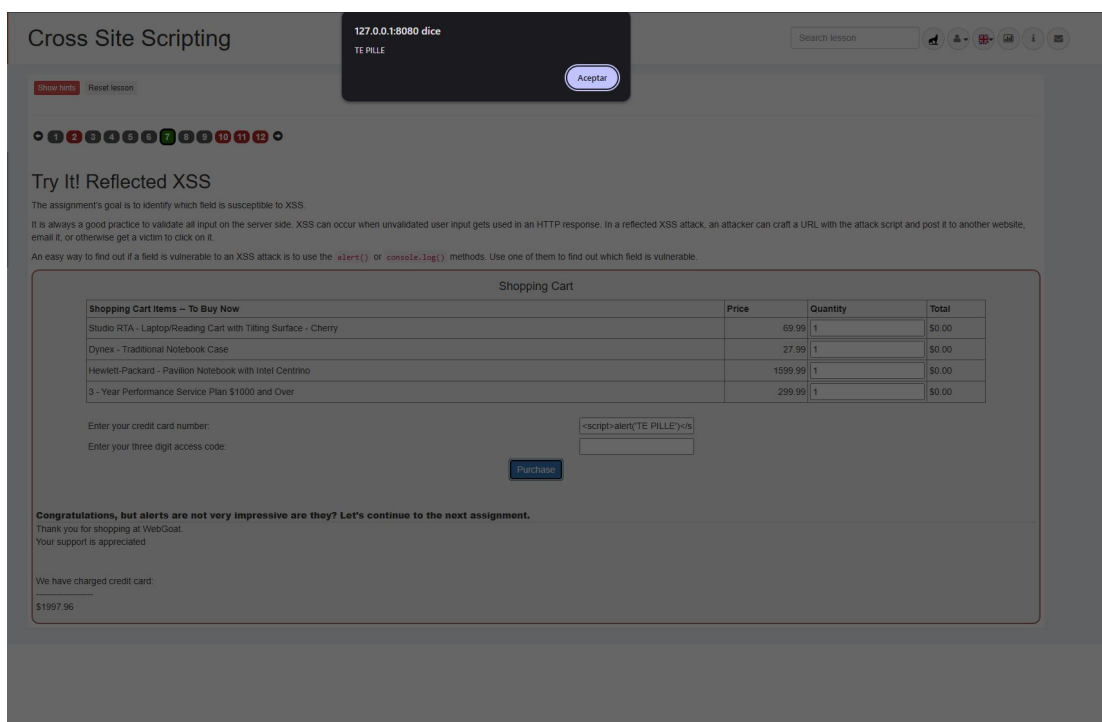


Figura 4.3: Ejecución de código JavaScript arbitrario (alert) en el navegador.

Criticidad: MEDIA / ALTA Permite ataques contra los usuarios de la web, como el **robo de cookies de sesión** (Session Hijacking), redirecciones a sitios de phishing o captura de teclas (Keylogging).

Mitigación y Recomendación

- **Técnica:** Aplicar **Codificación de Salida (Output Encoding)** dependiente del contexto.
- **Explicación:** El servidor debe convertir los caracteres especiales (como `<` y `>`) en sus equivalentes seguros HTML (`<` y `>`) antes de enviarlos al navegador. Así, el navegador muestra el texto pero no lo ejecuta como código.
- **Referencia OWASP:** [XSS Prevention Cheat Sheet](#)

4.4 A5: Security Misconfiguration (XXE Injection)

Descripción del Fallo El analizador XML de la aplicación está mal configurado y permite la definición de Entidades Externas (XXE). Esto permite a un atacante incluir archivos locales del servidor en la respuesta de la aplicación.

Proceso de Explotación Se interceptó una petición de comentario con Burp Suite y se modificó el XML para definir una entidad maliciosa llamada &xxe; que apunta a la raíz del sistema de archivos (file:///).

Payload inyectado en la cabecera XML:

```
<!DOCTYPE root [<!ENTITY xxe SYSTEM "file:///"]>
```

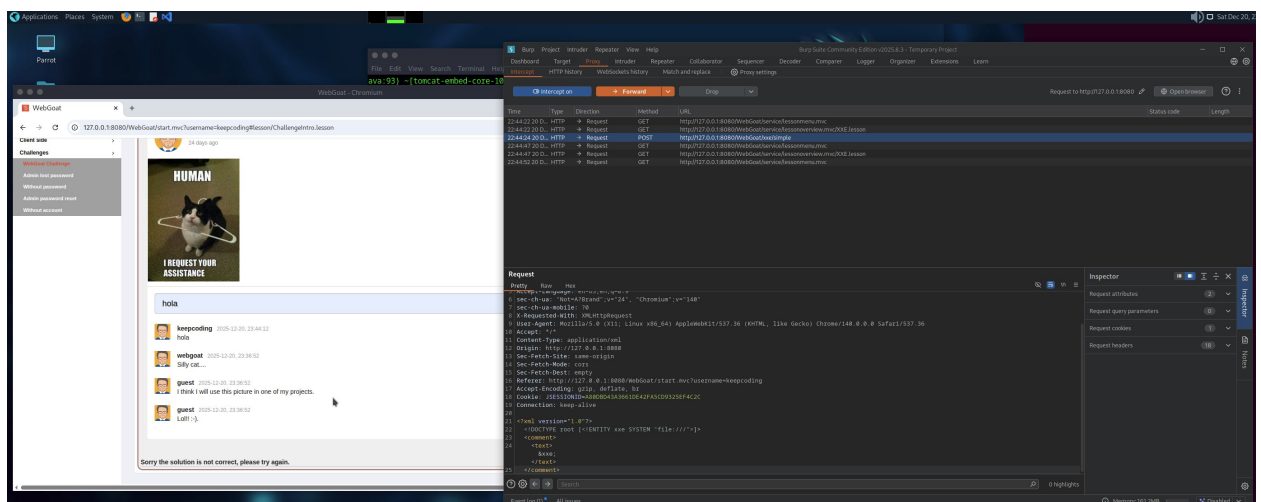


Figura 4.4.1: Interceptación y modificación de la petición XML en Burp Suite.

Evidencia del Resultado Al enviar la petición modificada, el servidor procesó la entidad externa y devolvió el listado de directorios de la raíz del sistema operativo Linux (/bin, /etc, /home, etc.) dentro del comentario publicado.

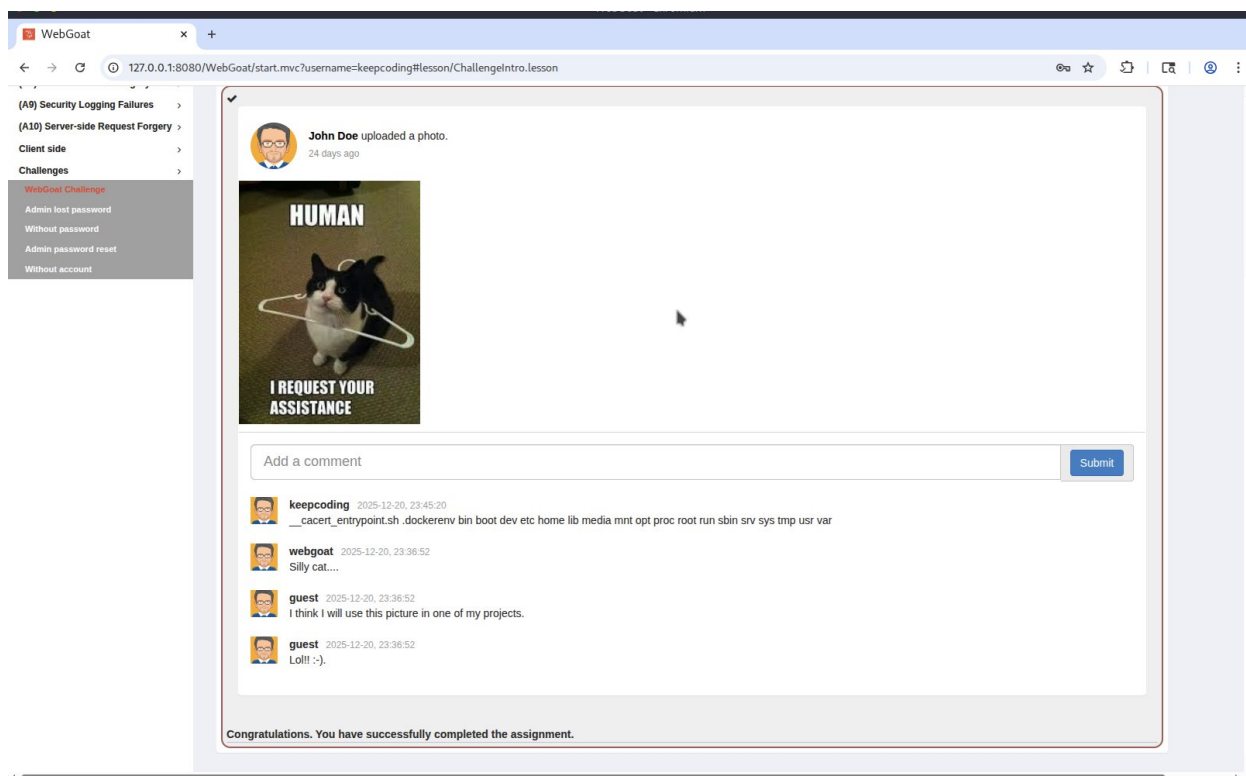


Figura 4.4.2: Respuesta del servidor exponiendo la estructura de directorios del sistema operativo.

Criticidad: ALTA Es un fallo de **Divulgación de Información**. Permite leer archivos de configuración (/etc/passwd), código fuente o claves SSH, facilitando la toma de control total del servidor (Remote Code Execution en algunos casos).

Mitigación y Recomendación

- **Técnica:** Deshabilitar el procesamiento de **DTDs (Document Type Definitions)** y entidades externas en el parser XML.
- **Explicación:** Se debe configurar el lector XML (como SAX o DOM parser) para que rechace explícitamente cualquier instrucción que intente cargar recursos externos.
- **Referencia OWASP:** [XXE Prevention Cheat Sheet](#)

4.5 A6: Componentes Vulnerables y Obsoletos

Descripción del Fallo La aplicación utiliza librerías de terceros que no han sido actualizadas y contienen vulnerabilidades públicas conocidas (CVEs).

Proceso de Identificación Se detectó el uso de la librería jquery-ui versión 1.10.4. Se comprobó mediante la documentación del laboratorio que esta versión es susceptible a ataques de Cross-Site Scripting (XSS), mientras que la versión 1.12.0 ya parchea este fallo.

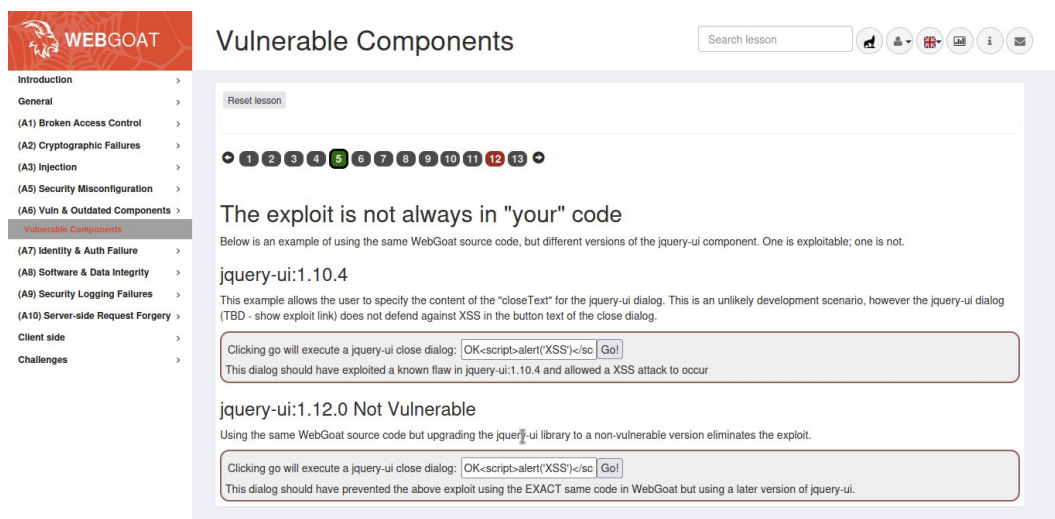


Figura 4.5: Comparativa entre la versión vulnerable (1.10.4) y la segura (1.12.0).

Criticidad: MEDIA El uso de componentes con vulnerabilidades conocidas facilita el trabajo a los atacantes, ya que existen *exploits* públicos listos para usar. Incrementa la superficie de ataque innecesariamente.

Mitigación y Recomendación

- **Técnica:** Implementar un proceso de **SCA (Software Composition Analysis)** y gestión de parches.
- **Explicación:** Mantener un inventario actualizado de todas las librerías usadas y aplicar actualizaciones de seguridad tan pronto como son publicadas por los fabricantes.
- **Referencia OWASP:** [Vulnerable and Outdated Components](#)

4.6 A7: Fallos de Identificación y Autenticación

Descripción del Fallo Los mecanismos de recuperación de contraseña son débiles. Las "preguntas de seguridad" utilizadas son predecibles o de conocimiento público (Ingeniería Social).

Proceso de Bypass Se intentó recuperar la contraseña del usuario administrador (**admin**). El sistema solicitó "¿Cuál es tu color favorito?". Mediante un ataque de diccionario simple (fuerza bruta manual), se probó con "**green**" (verde), logrando el acceso inmediato y permitiendo el cambio de contraseña.

The screenshot shows the WebGoat interface with the 'Password reset' page. On the left is a navigation menu with categories like Introduction, General, and Authentication Bypasses. The 'Password reset' link is highlighted. The main content area has a 'Reset lesson' button and a progress indicator showing 7 steps, with step 4 (Security questions) being the current one. Below this, there is an 'Assignment' section with a form titled 'WebGoat Password Recovery'. The form asks for 'Your username' (admin) and 'What is your favorite color?' (green). A 'Submit' button is at the bottom. A message at the bottom of the form says 'Congratulations. You have successfully completed the assignment.'

Recuperación exitosa de la cuenta de administrador mediante respuesta predecible.

Criticidad: MUY ALTA El compromiso de una cuenta administrativa otorga control total sobre la aplicación. Las preguntas de seguridad no deben usarse como único factor de autenticación.

Mitigación y Recomendación

- **Técnica:** Implementar **Autenticación Multifactor (MFA/2FA)** y eliminar las preguntas de seguridad.
- **Explicación:** Reemplazar las preguntas por el envío de un código temporal (OTP) al correo o móvil del usuario verificado. Esto asegura que solo el dueño legítimo del dispositivo pueda recuperar la cuenta.
- **Referencia OWASP:** [Authentication Cheat Sheet](#)

5. CONCLUSIONES Y CIERRE

5.1 Resumen de conclusiones finales La auditoría realizada sobre **WebGoat 8.1.0** ha evidenciado graves deficiencias de seguridad en el ciclo de desarrollo de software. La aplicación carece de controles básicos de validación de entrada, lo que la hace altamente susceptible a inyecciones (SQL, XXE, XSS).

5.2 Cuadro Resumen de Riesgos

Vulnerabilidad	Nivel de Riesgo	Impacto de Negocio
Inyección SQL	CRÍTICA	Robo total de datos y fraude financiero.
Fallos de Autenticación	MUY ALTA	Secuestro de cuentas de administrador.
XXE Injection	ALTA	Exposición de archivos del servidor.
XSS Reflejado	MEDIA/ALTA	Ataques a clientes y robo de sesiones.
Componentes Obsoletos	MEDIA	Exposición a exploits públicos conocidos.

5.3 Análisis de Post-explotación Se ha confirmado que vulnerabilidades como **XXE** e **Inyección SQL** no solo afectan a la aplicación web, sino que podrían servir como pivote para comprometer el servidor subyacente y la red interna de la organización. Es imperativo abordar las recomendaciones de este informe de manera inmediata, priorizando **la implementación de Consultas Parametrizadas y MFA**.

6. REFERENCIAS Y FUENTES TÉCNICAS

Para facilitar la labor del equipo de desarrollo en la remediación de los hallazgos, se adjuntan las guías oficiales de OWASP utilizadas como referencia en esta auditoría:

1. **Prevención de Inyección SQL:**
 - *Enlace:*
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
1. **Prevención de Cross-Site Scripting (XSS):**
 - *Enlace:*
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
1. **Prevención de XML External Entity (XXE):**
 - *Enlace:*
https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html
1. **Guía de Autenticación Segura:**
 - *Enlace:*
https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
1. **Gestión de Componentes Vulnerables:**
 - *Enlace:*
https://owasp.org/www-project-top-ten/2017/A9_2017-Using_Components_with_Known_Vulnerabilities

